Лабораторна робота №5

Тема роботи

Реалізація інтерактивних компонентів веб-застосунку засобами фреймворку Vue.js 3 (Options API, JavaScript) без використання додаткових бібліотек, на основі підключення через CDN

Мета роботи

Ознайомитися з концепцією компонентного підходу до побудови веб-інтерфейсів; набути практичних навичок роботи з фреймворком Vue.js 3 через Options API на мові JavaScript; освоїти реактивність даних, директиви шаблонів, обчислювані властивості (computed), спостерігачі (watch) та механізм передачі даних між компонентами через props та події ($emit); навчитися підключати Vue.js через CDN без збірника.

Теоретичні відомості

Vue.js - це прогресивний JavaScript-фреймворк для побудови користувацьких інтерфейсів. У цій лабораторній роботі використовується Vue.js 3 із підходом *Options API*, що є найбільш доступним для початківців та не потребує збірника. Підключення здійснюється через CDN одним тегом: `<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>`. Застосунок монтується методом `Vue.createApp({...}).mount("#app")`, де `#app` - ідентифікатор кореневого HTML-елемента.

Options API організовує логіку компонента через об'єкт опцій. Секція `data()` повертає реактивний об'єкт із початковим станом компонента - будь-яка зміна властивостей цього об'єкта автоматично оновлює DOM. Секція `methods` містить функції-обробники, доступні з шаблону. Секція `computed` описує похідні значення, що автоматично перераховуються при зміні залежних реактивних даних та кешуються між викликами. Секція `watch` дозволяє реагувати на зміну конкретної реактивної властивості з виконанням побічних ефектів.

Директиви Vue - це спеціальні атрибути HTML з префіксом `v-`, що надають шаблону реактивну поведінку. Основні директиви: `v-bind` (або скорочення `:`) - реактивне прив'язування атрибутів; `v-on` (або `@`) - реєстрація обробників подій; `v-model` - двостороннє зв'язування даних з елементами форм; `v-if` / `v-else` / `v-show` - умовне відображення; `v-for` - рендер списків з обов'язковим атрибутом `:key`.

Компонентний підхід передбачає розбиття інтерфейсу на незалежні, повторно використовувані частини. Дочірній компонент реєструється через секцію `components: {}` батьківського застосунку. Батько передає дані у компонент через *props* (властивості, що оголошуються у секції `props: []` або `props: {}`). Дочірній компонент не може змінювати props безпосередньо - натомість він сповіщає батька через *користувацькі події* методом `this.$emit("назва-події", дані)`. Батько підписується на них через директиву `@назва-події="метод"`.

Хуки життєвого циклу (*lifecycle hooks*) дозволяють виконати код на певному етапі існування компонента. Найбільш вживані: `created()` - компонент створено, DOM ще не існує (підходить для завантаження даних); `mounted()` - компонент змонтовано в DOM (підходить для `setInterval`, звернення до DOM-елементів); `beforeUnmount()` - перед знищенням (підходить для `clearInterval`). Доступ до DOM-елемента зі шаблону здійснюється через атрибут `ref="назва"` та `this.$refs.назва`.

Вхідні дані для виконання роботи

- Виконані Лабораторна робота №1 (index.html), Лабораторна робота №2 (styles.css) та Лабораторна робота №4 (script.js).
- Знання основ JavaScript: масиви, об'єкти, функції, стрілкові функції, деструктуризація.
- Браузер з інструментами розробника; рекомендується встановити розширення Vue DevTools.
- Варіант завдання відповідно до номера студента у списку групи (Додаток А).

Порядок виконання роботи

Варіант завдання визначається відповідно до номера студента у списку групи (Додаток А). Vue-компонент, обов'язкові директиви та обов'язкова

концепція Vue задаються індивідуально. Тема сторінки відповідає темі попередніх лабораторних робіт.

Для виконання лабораторної роботи необхідно досягти наступні пункти:

1. Підготовка файлів: створити папку lab5. Скопіювати index.html та styles.css з попередніх лабораторних робіт. Підключити Vue.js 3 через CDN тегом <script> у розділі <head> до підключення власних скриптів. Створити файл app.js для основного застосунку та компоненти у окремих файлах з розширенням .js (наприклад, components/CardItem.js). Усі скрипти підключити з атрибутом type="module".
2. Кореневий застосунок Vue: у файлі app.js створити кореневий застосунок через Vue.createApp({}) та змонтувати його на елемент з id="app" у HTML. Визначити початковий стан у секції data(): масив об'єктів з даними відповідно до теми варіанта (щонайменше 6 записів). Усі дані зберігати виключно в реактивному стані - не використовувати глобальні змінні поза об'єктом застосунку.
3. Шаблон та директиви: розмістити кореневий елемент у HTML та наповнити його шаблоном Vue. Застосувати усі директиви, вказані у варіанті (Додаток А). Для рендеру списків обов'язково вказувати атрибут :key з унікальним ідентифікатором. Використовувати скорочення : замість v-bind: та @ замість v-on: для стислості коду.
4. Computed-властивості: реалізувати обчислювані властивості (секція computed) відповідно до варіанта (Додаток А). Computed не повинні містити побічних ефектів - лише обчислення та повернення значення. Перевірити у Vue DevTools, що computed оновлюється реактивно при зміні залежних даних.
5. Дочірній компонент: виділити логічно завершену частину інтерфейсу в окремий компонент (наприклад, картка товару, рядок таблиці). Компонент оголосити як об'єкт з опціями у окремому .js-файлі та зареєструвати локально через секцію components батьківського застосунку. Визначити необхідні props з валідацією типу (props: { title: String, price: Number }). Передавати дані через $emit відповідно до варіанта (Додаток А).
6. Watch та методи: реалізувати спостерігачі (секція watch) відповідно до варіанта (Додаток А). Кожен watch повинен мати чітко визначену відповідальність. Методи (секція methods) іменувати дієсловами у

camelCase (addItem, removeItem, toggleFilter). Уникати дублювання логіки між methods та computed.

7. Стилізація реактивних станів: використовувати директиву :class з об'єктним синтаксисом ({ "клас": умова }) або масивом для динамічного застосування CSS-класів залежно від стану даних. Активний елемент, вибраний товар, заповнене поле тощо повинні мати візуальну індикацію через CSS-класи, керовані Vue.
8. Тестування та документування: перевірити роботу застосунку у Vue DevTools (реактивність даних, props, події). Перевірити консоль браузера на відсутність помилок та попереджень Vue. Протестувати граничні випадки (порожній список, некоректне введення, максимальна кількість елементів). Зробити скриншоти для звіту, що демонструють різні стани компонента та дерево компонентів у Vue DevTools.

Подання матеріалів на захист роботи

До захисту лабораторної роботи подаються наступні артефакти:

- Файли index.html, styles.css, app.js та файли компонентів (папка components/) з реалізованим Vue-застосунком відповідно до варіанта.
- Звіт у форматі PDF або Google Docs: опис архітектури компонентів (схема батько–діти), скриншоти різних станів застосунку, скриншот Vue DevTools з деревом компонентів та реактивним станом.
- Посилання на репозиторій GitHub або архів ZIP з вихідними файлами проекту.

Контрольні питання

1. Що таке реактивність у Vue.js? Як Vue відстежує зміни даних та автоматично оновлює DOM?
2. У чому різниця між computed-властивістю та методом (method)? Коли доцільно використовувати кожен підхід?
3. Поясніть механізм передачі даних між компонентами через props та $emit. Чому дочірній компонент не повинен змінювати props безпосередньо?
4. Для чого використовується директива v-model? Як вона реалізована "під капотом" через v-bind та v-on?
5. Чим відрізняються директиви v-if та v-show? У яких ситуаціях доцільно використовувати кожну?

6. Навіщо вказувати атрибут :key при використанні директиви v-for? Що відбудеться, якщо його не вказати?

Додаток А

Варіанти завдань до Лабораторної роботи №5

Варіант	Тема (з Лаб. №1–4)	Vue-компонент	Обов'язкові директиви	Обов'язкова концепція Vue
1	Інтернет-магазин електроніки	Каталог товарів з фільтром та кошиком	`v-for, v-if, v-model, v-on`	computed для фільтрації списку; $emit('add-to-cart') з дочірнього компонента картки
2	Портфоліо фотографа	Інтерактивна галерея з вибором фото	`v-for, v-bind:class, v-on:click, v-show`	data() для активного індексу; computed для поточного фото
3	Блог про подорожі	Форма підписки на блог з валідацією	`v-model, v-on:submit, v-if, v-bind:disabled`	watch для поля email з перевіркою формату; computed для стану кнопки
4	Сайт ресторану	Меню ресторану з вибором страв та підрахунком замовлення	`v-for, v-model, v-on, v-bind:class`	computed для підсумку замовлення; компонент MenuItem з props та $emit
5	Освітній портал	Пошук курсів з живою фільтрацією	`v-model, v-for, v-if, v-bind`	computed для фільтрованого списку курсів на основі рядка пошуку та категорії
6	Новинний сайт	Стрічка новин з пагінацією	`v-for, v-if, v-on:click, v-bind:class`	computed для поточної сторінки; data() для номера сторінки та розміру сторінки
7	Сайт спортивного клубу	Трекер тренувань (додати / видалити запис)	`v-for, v-model, v-on:submit, v-on:click`	methods для додавання та видалення; computed для загальної тривалості тренувань
8	Медичний центр	Форма запису на прийом з підтвердженням	`v-model, v-on:submit, v-if, v-bind:disabled`	watch для поля дати (заборона минулих дат); багатокроковий v-if для станів форми
9	Туристичне агентство	Каталог турів з фільтром за ціною та напрямком	`v-for, v-model, v-if, v-bind`	computed з подвійним фільтром (ціна + напрямок); компонент TourCard з props
10	IT-компанія	Портфоліо проектів з сортуванням та фільтром	`v-for, v-model, v-on:click, v-bind:class`	computed для відфільтрованого та відсортованого масиву; data() для активного фільтра
11	Інтернет-магазин одягу	Конфігуратор товару (розмір, колір, кількість)	`v-model, v-for, v-bind:class, v-on`	computed для фінальної ціни; watch для скидання кількості при зміні розміру
12	Сайт кінотеатру	Розклад сеансів з фільтром за жанром	`v-for, v-model, v-if, v-bind:class`	computed для фільтрованих сеансів; компонент SessionCard з props та slot

13	Музична школа	Таймер репетиції зі стартом/зупинкою	`v-on:click, v-bind:disabled, v-if`	data() для стану таймера; методи start/stop з setInterval; computed для форматованого часу
14	Бібліотека	Пошук книг у каталозі з деталями	`v-model, v-for, v-if, v-on:click`	computed для результатів пошуку; v-if для відображення деталей обраної книги
15	Фітнес-центр	Щоденник тренувань (CRUD)	`v-for, v-model, v-on, v-if`	methods для create/update/delete; computed для статистики (кількість, загальний час)
16	Автосалон	Калькулятор вартості авто з опціями	`v-model, v-for, v-bind, v-on`	computed для динамічного перерахунку ціни; watch для скидання опцій при зміні моделі
17	Готель	Форма бронювання номера з перевіркою дат	`v-model, v-on:submit, v-if, v-bind:disabled`	computed для кількості ночей та загальної вартості; watch для валідації дат
18	Кулінарний блог	Книга рецептів з пошуком та фільтром за часом	`v-model, v-for, v-if, v-bind:class`	computed з комбінованим фільтром (текст + час приготування); компонент RecipeCard з props
19	Сайт університету	Журнал відвідуваності занять	`v-for, v-model, v-on:click, v-bind:class`	data() для масиву студентів; computed для відсотка присутніх; $emit для позначення
20	Галерея мистецтва	Слайдер творів мистецтва з навігацією	`v-bind:src, v-on:click, v-if, v-bind:class`	data() для поточного індексу; computed для попереднього/наступного; методи prev/next
21	Веб-студія	Калькулятор вартості послуг веб-студії	`v-model, v-for, v-bind, v-on`	computed для підсумку вибраних послуг; компонент ServiceItem з props та $emit
22	Юридична компанія	Акордеон FAQ з пошуком	`v-for, v-model, v-on:click, v-show, v-if`	computed для фільтрованих запитань; data() для активного індексу акордеону
23	Будівельна компанія	Трекер етапів будівельного проекту	`v-for, v-on:click, v-bind:class, v-if`	computed для відсотка виконання; $emit('complete-step') з компонента StageItem
24	Сайт стартапу	Таймер зворотного відліку до події	`v-bind, v-if, v-on:click`	data() для цільової дати; computed для залишку днів/годин/хвилин/секунд; setInterval у mounted()
25	Медіа-агентство	Редактор тегів публікації	`v-model, v-for, v-on, v-if`	methods для додавання та видалення тегів; computed для перевірки дублікатів та ліміту
26	Інтернет-магазин книг	Кошик книгарні з підрахунком	`v-for, v-model, v-on:click, v-if`	computed для загальної суми та кількості; $emit('remove') та $emit('update-qty') з CartItem
27	Благодійний фонд	Форма збору пожертв з прогрес-індикатором	`v-model, v-bind:style, v-if, v-on:submit`	computed для відсотка від мети та стилю прогрес-бару; watch для валідації суми

28	Ветеринарна клініка	Журнал вакцинацій тварин (CRUD-таблиця)	`v-for`, `v-model`, `v-on`, `v-if`	methods для додавання, редагування та видалення рядків; computed для сортування за датою
29	Дитячий навчальний центр	Інтерактивний розклад занять з перетягуванням пріоритету	`v-for`, `v-on:click`, `v-bind:class`, `v-if`	methods для переміщення елементів вгору/вниз у масиві; computed для валідності розкладу
30	Наукова лабораторія	Панель результатів досліджень з фільтром та сортуванням	`v-for`, `v-model`, `v-on`, `v-bind:class`	computed з ланцюжком filter().sort(); компонент ResultRow з props; watch для скидання сторінки